

DATA STRUCTURES AND ALGORITHMS

Linked Lists

LIST ADT

- What we mean by a "list".
- It is a collection of some *elements*.
- The most important concept related to lists is that of *position*. In other words, we perceive that there is a first element in the list, a second element, and so on.
- So, we consider a *list* to be a finite, ordered sequence of data items known as *elements*.
- This is close to the mathematical concept of a *sequence*.

LIST ADT

- "Ordered" in this definition means that each element has a position in the list.
- So the term "ordered" in this context does **not** mean that the list elements are sorted by value.
- Each list element must have some data type.
- Although there is no conceptual objection to lists whose elements have differing data types if the application requires it.

LIST ADT

- A list is said to be ***empty*** when it contains no elements.
- The number of elements currently stored is called the ***length*** of the list.
- The beginning of the list is called the ***head***, the end of the list is called the ***tail***.

Basic Operations on LIST ADT

- A list should be able to grow and shrink in size as we insert and remove elements.
- We should be able to insert and remove elements from anywhere in the list.
- We should be able to gain access to any element's value, either to read it or to change it.
- We must be able to create and clear (or reinitialize) lists.
- It is also convenient to access the next or previous element from the "current" one.

Implementations of LIST ADT

- There are two standard approaches to implementing lists:
 - the *array-based list*,
 - and the *linked list*.
- We have already learnt about arrays in detail.
- Now, we will discuss about:
 - What linked-list is?
 - What are its types?

Need for Linked List

- Sometimes you need to store a list of elements in memory.
 - Suppose you're writing an app to manage your to-do's.
 - You'll want to store the to-do's as a list in memory.
 - Let's store the to-do's in an array.
- Using an array means all your tasks are stored contiguously (right next to each other) in memory.



Need for Linked List

- Now suppose you want to add a fourth task.
 - But the next location is taken up by someone else's stuff!
 - You have to move to a new spot where you all fit.
- In this case, you need to ask your computer for a different chunk of memory that can fit four tasks.
- Then you need to move all your tasks there.



Need for Linked List

- Similarly, adding new items to an array can be a big pain.
- If you're out of space and need to move to a new spot in memory every time, adding a new item will be really slow.
- One easy fix is to “hold seats”: even if you have only 3 items in your task list, you can ask the computer for 10 slots.
- Then you can add 10 items to your task list without having to move.



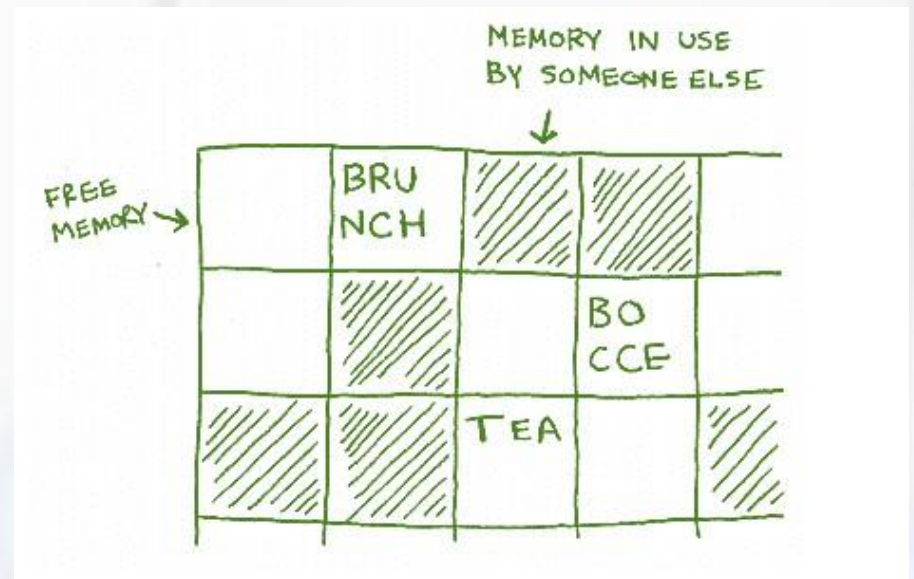
Need for Linked List

- This is a good workaround, but you should be aware of a couple of downsides:
 - *You may not need the extra slots that you asked for, and then that memory will be wasted. You aren't using it, but no one else can use it either.*
 - *You may add more than 10 items to your task list and have to move anyway.*



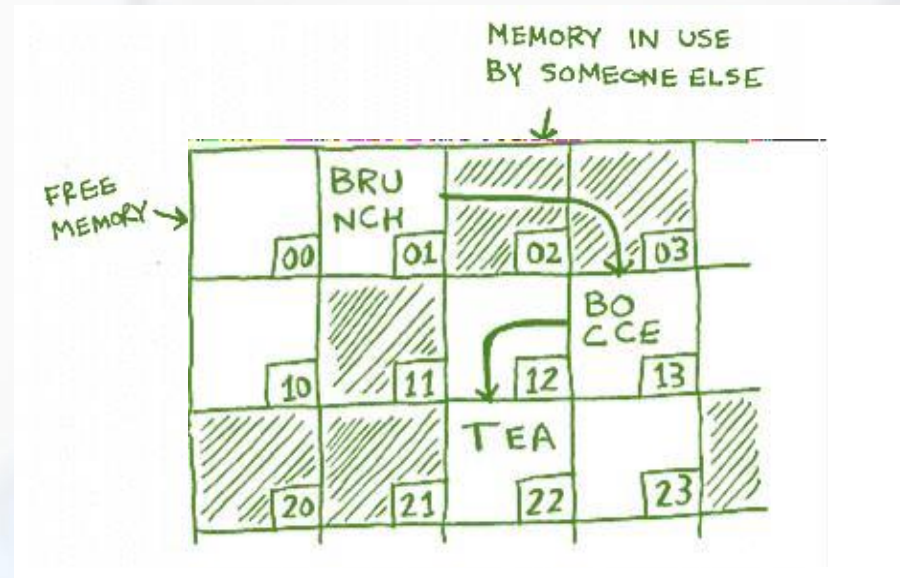
Need for Linked List

- Linked lists solve this problem.
 - With linked lists, your items can be anywhere in memory.
 - Each item stores the address of the next item in the list.
- A bunch of random memory addresses are linked together.



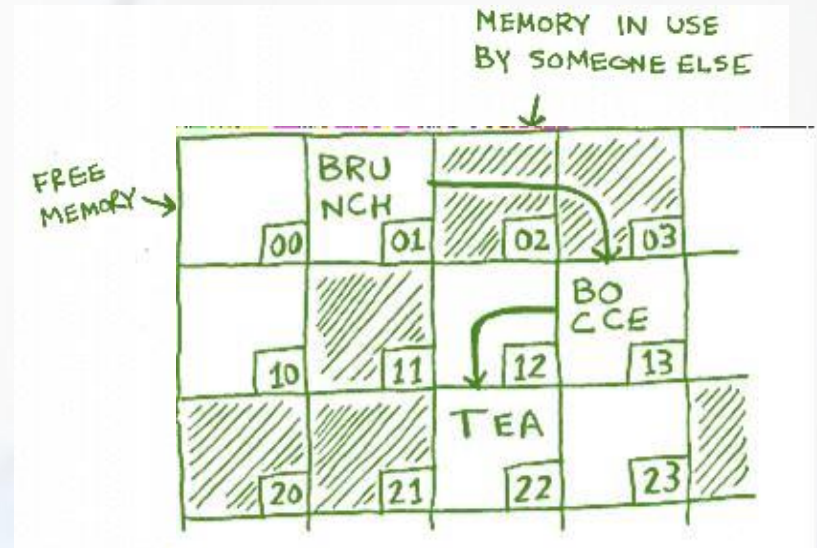
Need for Linked List

- Linked lists solve this problem.
 - With linked lists, your items can be anywhere in memory.
 - Each item stores the address of the next item in the list.
- A bunch of random memory addresses are linked together.



Need for Linked List

- You go to the first address, and it says, “The next item can be found at address 123.”
- So you go to address 123, and it says, “The next item can be found at address 847,” and so on.
- With linked lists, you never have to move your items.



Linked Lists

1. Concepts of linked lists
2. Simple linked lists
3. Circular linked lists
4. Double linked lists
5. Circular double linked lists

Concepts of linked list

- A linked list is a linear collection of data elements called nodes in which linear representation is given by links from one node to the next node.
- Similar to array, it is a linear collection of data elements of the **same type**.
- Different from array, data elements of linked list are generally not lined in consecutive memory space; instead they are **dispersed** in various locations

Concepts of linked list

- Linked list is a data structure which in turn can be used to implement other data structures. Thus, it acts as building block to implement data structures like stacks, queues and their variations.
- A linked list can be perceived as a train or a sequence of nodes in which each node contains one or more data fields and a pointer to the next node.

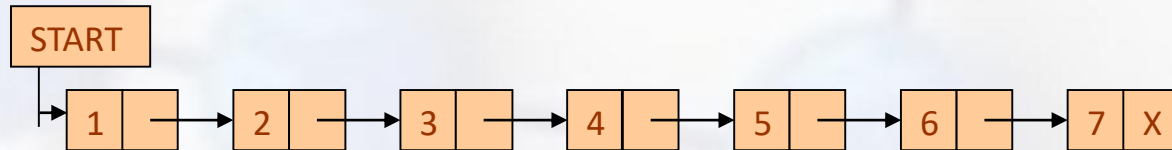
Element of linked list

- Linked list element (node) is user defined structure data type, typically contains two parts
 - data variables
 - pointers to next elements, hold the addresses of next elements

Example:

```
struct node {  
    int data;           // data  
    struct node *next; // pointer to next element  
};
```

Simple Linked List



- In the above linked list, every node contains two parts - one integer and the other a pointer to the next node.
- The data part of the node which contains data may include a simple data type, an array or a structure.
- The pointer part of the node contains a pointer to the next node (or address of the next node in sequence).
- The last node will have no next node connected to it, so it will store a NULL value.
- A linked list is defined by the pointer pointing to the first node, e.g START

Linked List Operations

1. Create a node and linked list
2. Traversal, e.g. display all elements
3. Search node
4. Insert node
at beginning, at end, after/before a node
5. Delete node
at beginning, at end, after/before a node
6. Sort

How to create nodes

A node is a structure data type, can be created in two methods, statically and dynamically.

- Static method
 - use array of structures
 - declared as globally outside functions
 - declared locally within a function
- Dynamic method (mostly used for linked list)
 - use stdlib function `malloc(size)` get memory space

```
struct node *np = (struct node*) malloc(sizeof(struct node));
```

How to create nodes

```
struct node *np = (struct node*) malloc(sizeof(struct node));
```

At run time, OS allocates consecutive sizeof(struct node) bytes in the heap region,

return the address of the address of the first memory cell,

store the address to struct node type pointer np.

Need (struct node*) to cast the return address to struct node pointer value.

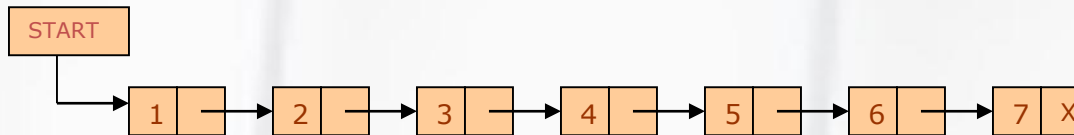
Need use free(np) to release when deleting the node!!!
Otherwise cause memory leaking

Traversing Linked Lists

- We can traverse the entire linked list using a single pointer variable called START.
- The START node contains the address of the first node; the next part of the first node in turn stores the address of its succeeding node.
- Using this technique the individual nodes of the list will form a chain of nodes.
- If $START = NULL$, this means that the linked list is empty and contains no nodes.

Singly Linked Lists

- A singly linked list is the simplest type of linked list in which every node contains some data and a pointer to the next node of the same data type.



Example:

```
struct node {  
    int data;  
    struct node *next;  
};
```

Traversal Singly Linked Lists

ALGORITHM FOR TRAVERSING A LINKED LIST

```
Step 1: [INITIALIZE] SET PTR = START
Step 2: Repeat Steps 3 and 4 while PTR != NULL
Step 3: Apply Process to PTR->DATA
Step 4: SET PTR = PTR->NEXT
        [END OF LOOP]
Step 5: EXIT
```

```
void display(struct node *ptr) {
    while(ptr != NULL) {
        printf("%d ", ptr->data); // process
        ptr = ptr->next;
    }
}
```

Call as display(START);

C++ CODE

- `#include <iostream>`
- `using namespace std;`
-
- `// Define a node of the linked list`
- `struct Node {`
- `int data;`
- `Node* next;`
- `};`
-
- `// Function to traverse and print the linked list`
- `void traverse(Node* start) {`
- `Node* ptr = start; // Step 1: Initialize PTR = START`
-
- `// Step 2: Repeat while PTR != NULL`
- `while (ptr != NULL) {`
- `cout << ptr->data << " "; // Step 3: Process PTR->DATA`
- `ptr = ptr->next; // Step 4: PTR = PTR->NEXT`
- `}`
- `cout << endl;`
- `}`



```
// Utility function to add a new node at the end
void insertEnd(Node*& start, int value) {
    Node* newNode = new Node();
    newNode->data = value;
    newNode->next = NULL;

    if (start == NULL) {
        start = newNode;
        return;
    }

    Node* temp = start;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
}

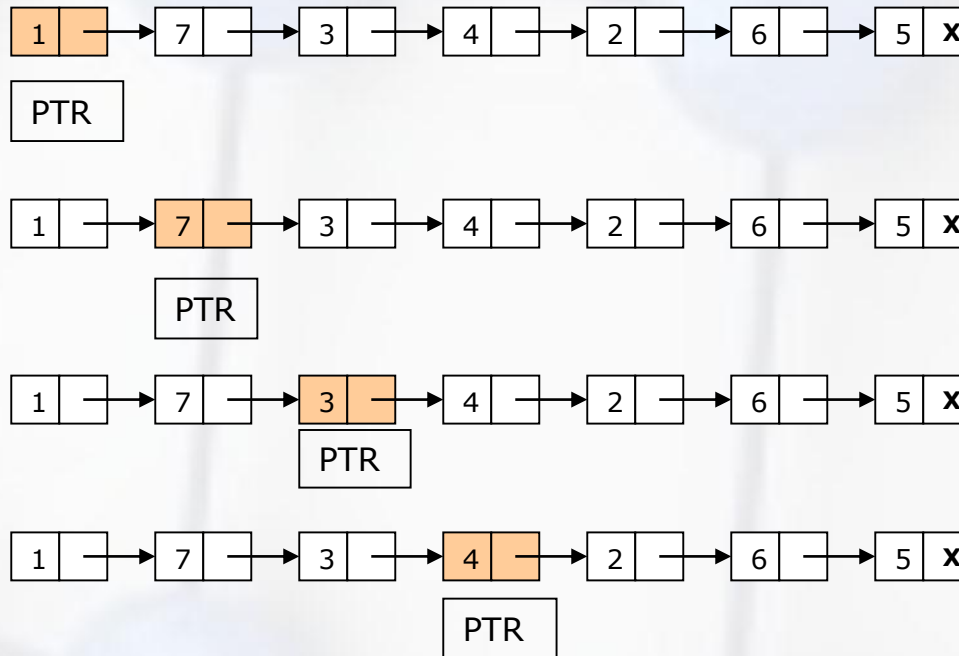
// Main function
int main() {
    Node* start = NULL;

    // Insert some nodes
    insertEnd(start, 10);
    insertEnd(start, 20);
    insertEnd(start, 30);
    insertEnd(start, 40);

    cout << "Traversing the linked list: ";
    traverse(start);

    return 0;
}
```

Searching for Val 4 in Linked List



Searching a Linked List

ALGORITHM TO SEARCH A LINKED LIST

Step 1: [INITIALIZE] SET PTR = START

Step 2: Repeat Step 3 while PTR != NULL

```
Step 3:   IF VAL = PTR->DATA
           SET POS = PTR
           Go To Step 5
        ELSE
           SET PTR = PTR->NEXT
        [END OF IF]
    [END OF LOOP]
```

Step 4: SET POS = NULL // not found

Step 5: EXIT // found, output POS

```
struct node* search(struct node *ptr, int num) {
    while((ptr != NULL) && (ptr->data != num)) {
        ptr = ptr->next;
    }
    return ptr;
}
// call example, search(START, 4)
```

C++ CODE

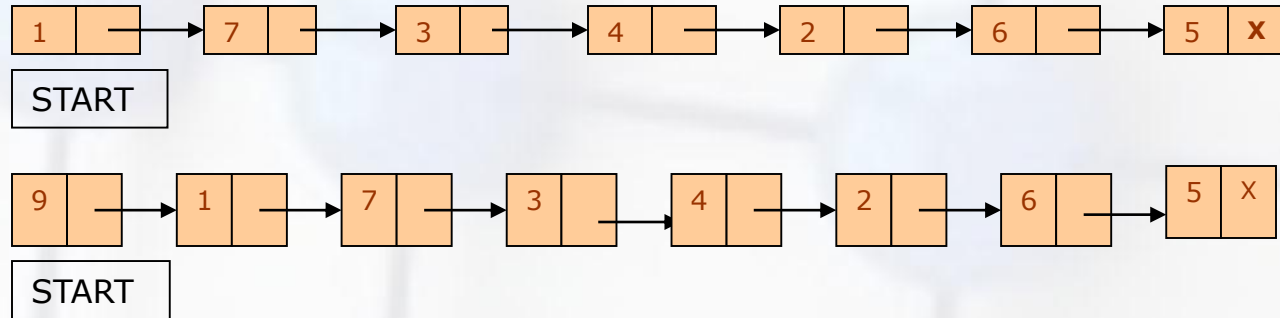
- `#include <iostream>`
- `using namespace std;`
-
- `// Define node structure`
- `struct Node {`
- `int data;`
- `Node* next;`
- `};`
-
- `// Function to search a value in the linked list`
- `Node* search(Node* start, int val) {`
- `Node* ptr = start; // Step 1: Initialize PTR = START`
-
- `// Step 2: Repeat while PTR != NULL`
- `while (ptr != NULL) {`
- `if (ptr->data == val) { // Step 3: If VAL == PTR->DATA`
- `return ptr; // Found → return address of node`
- `}`
- `ptr = ptr->next; // Else move to next node`
- `}`
-
- `return NULL; // Step 4: Not found`
- `}`

```

• // Utility function to insert a node at the end
• void insertEnd(Node*& start, int value) {
•     Node* newNode = new Node();
•     newNode->data = value;
•     newNode->next = NULL;
•
•     if (start == NULL) {
•         start = newNode;
•         return;
•     }
•     Node* temp = start;
•     while (temp->next != NULL) {
•         temp = temp->next;
•     }
•     temp->next = newNode;
• }
•
• // Main function
• int main() {
•     Node* start = NULL;
•
•     // Create linked list: 10 -> 20 -> 30 -> 40
•     insertEnd(start, 10);
•     insertEnd(start, 20);
•     insertEnd(start, 30);
•     insertEnd(start, 40);
•
•     // Search for a value
•     int num = 30;
•     Node* pos = search(start, num);
•
•     if (pos != NULL)
•         cout << "Value " << num << " found at node address: " << pos << endl;
•     else
•         cout << "Value " << num << " not found in the list." << endl;
•
•     return 0;
• }

```

Inserting a Node at the Beginning



ALGORITHM: INSERT A NEW NODE IN THE BEGINNING OF THE LINKED LIST

Step 1: IF AVAIL = NULL, then

Write OVERFLOW

Go to Step 7

[END OF IF]

Step 2: SET New_Node = AVAIL

Step 3: SET AVAIL = AVAIL->NEXT

Step 4: SET New_Node->DATA = VAL

Step 5: SET New_Node->Next = START

Step 6: SET START = New_Node

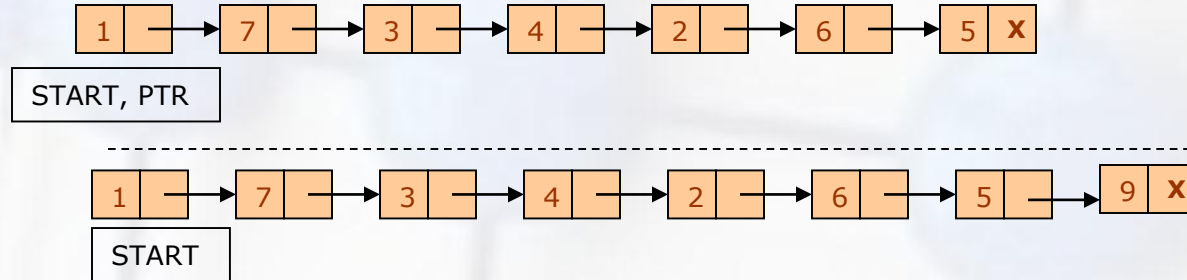
Step 7: EXIT

C++ CODE

- `#include <iostream>`
- `using namespace std;`
-
- `// Define node structure`
- `struct Node {`
- `int data;`
- `Node* next;`
- `};`
-
- `// Function to insert a node at the beginning`
- `void insertAtBeginning(Node*& start, int val) {`
- `// Step 1: Create a new node`
- `Node* newNode = new(nothrow) Node; // nothrow prevents crash if memory fails`
-
- `if (newNode == NULL) { // Step 1 check (Overflow)`
- `cout << "OVERFLOW: No memory available!" << endl;`
- `return;`
- `}`
-
- `// Step 2: Assign value to new node`
- `newNode->data = val;`
-
- `// Step 3: Point new node to current start`
- `newNode->next = start;`
-
- `// Step 4: Update start`
- `start = newNode;`
- `}`

- // Function to traverse and display linked list
- void traverse(Node* start) {
- Node* ptr = start;
- while (ptr != NULL) {
- cout << ptr->data << " ";
- ptr = ptr->next;
- }
- cout << endl;
- }
-
- // Main function
- int main() {
- Node* start = NULL; // Initially empty list
-
- // Insert nodes at the beginning
- insertAtBeginning(start, 30);
- insertAtBeginning(start, 20);
- insertAtBeginning(start, 10);
-
- cout << "Linked List after insertions: ";
- traverse(start); // Expected Output: 10 20 30
-
- return 0;
- }

Inserting a Node at the End



ALGORITHM TO INSERT A NEW NODE AT THE END OF THE LINKED LIST

Step 1: IF AVAIL = NULL, then

Write OVERFLOW

Go to Step 10

[END OF IF]

Step 2: SET New_Node = AVAIL

Step 3: SET AVAIL = AVAIL->NEXT

Step 4: SET New_Node->DATA = VAL

Step 5: SET New_Node->Next = NULL

Step 6: SET PTR = START

Step 7: Repeat Step 8 while PTR->NEXT != NULL

Step 8: SET PTR = PTR->NEXT

[END OF LOOP]

Step 9: SET PTR->NEXT = New_Node

Step 10: EXIT

Inserting a Node after Node that has Value NUM

ALGORITHM TO INSERT A NEW NODE AFTER A NODE THAT HAS VALUE NUM

Step 1: IF AVAIL = NULL, then
 Write OVERFLOW
 Go to Step 12
 [END OF IF]
Step 2: SET New_Node = AVAIL
Step 3: SET AVAIL = AVAIL->NEXT
Step 4: SET New_Node->DATA = VAL
Step 5: SET PTR = START
Step 6: SET PREPTR = PTR
Step 7: Repeat Steps 8 and 9 while PREPTR->DATA != NUM
Step 8: SET PREPTR = PTR
Step 9: SET PTR = PTR->NEXT
 [END OF LOOP]
Step 10: PREPTR->NEXT = New_Node
Step 11: SET New_Node->NEXT = PTR
Step 12: EXIT

C++ CODE

- `#include <iostream>`
- `using namespace std;`
-
- `// Node structure`
- `struct Node {`
- `int data;`
- `Node*#include <iostream>`
- `using namespace std;`
-
- `// Node structure`
- `struct Node {`
- `int data;`
- `Node* next;`
- `};`
-
- `// Function to get a node from AVAIL`
- `Node* getNode(Node*& AVAIL) {`
- `if (AVAIL == NULL) {`
- `cout << "OVERFLOW" << endl;`
- `return NULL;`
- `}`
- `Node* newNode = AVAIL;`
- `AVAIL = AVAIL->next; // move AVAIL forward`
- `newNode->next = NULL;`
- `return newNode;`
- `}`

- // Function to insert after a node with value NUM
- void insertAfter(Node*& START, Node*& AVAIL, int VAL, int NUM) {
- // Step 1–3: Take node from AVAIL
- Node* newNode = getNode(AVAIL);
- if (newNode == NULL) return; // no free nodes
-
- // Step 4: Assign data
- newNode->data = VAL;
-
- // Step 5–6: Traverse to find NUM
- Node* PTR = START;
- while (PTR != NULL && PTR->data != NUM) {
- PTR = PTR->next;
- }
-
- // Step 7: If NUM not found
- if (PTR == NULL) {
- cout << "Node with value " << NUM << " not found." << endl;
- // Return node back to AVAIL
- newNode->next = AVAIL;
- AVAIL = newNode;
- return;
- }

- `// Step 8–9: Insert after NUM`
- `newNode->next = PTR->next;`
- `PTR->next = newNode;`
- `}`
-
- `// Function to display linked list`
- `void display(Node* start) {`
- `Node* temp = start;`
- `while (temp != NULL) {`
- `cout << temp->data << " -> ";`
- `temp = temp->next;`
- `}`
- `cout << "NULL" << endl;`
- `}`
-
- `// Function to initialize AVAIL with free nodes`
- `Node* initializeAvail(int n) {`
- `Node* head = NULL;`
- `for (int i = 0; i < n; i++) {`
- `Node* temp = new Node;`
- `temp->next = head;`
- `head = temp;`
- `}`
- `return head;`

- // Main function
- int main() {
- Node* START = NULL;
- Node* AVAIL = initializeAvail(10); // pool of 10 free nodes
-
- // Manually build a linked list: 10 -> 20 -> 30
- Node* n1 = getNode(AVAIL);
- n1->data = 10;
- START = n1;
-
- Node* n2 = getNode(AVAIL);
- n2->data = 20;
- n1->next = n2;
-
- Node* n3 = getNode(AVAIL);
- n3->data = 30;
- n2->next = n3;
- n3->next = NULL;
-
- cout << "Original List: ";
- display(START);
-
- // Insert 25 after 20
- insertAfter(START, AVAIL, 25, 20);
-
- cout << "After inserting 25 after 20: ";
- display(START);
-
- // Try inserting after non-existing node
- insertAfter(START, AVAIL, 40, 50);
-
- cout << "Final List: ";
- display(START);
-
- return 0;
- }

Deleting the First Node

Algorithm to delete the first node from the linked list

Step 1: IF START = NULL, then
 Write UNDERFLOW
 Go to Step 5
[END OF IF]

Step 2: SET PTR = START

Step 3: SET START = START->NEXT

Step 4: FREE PTR

Step 5: EXIT



START



START

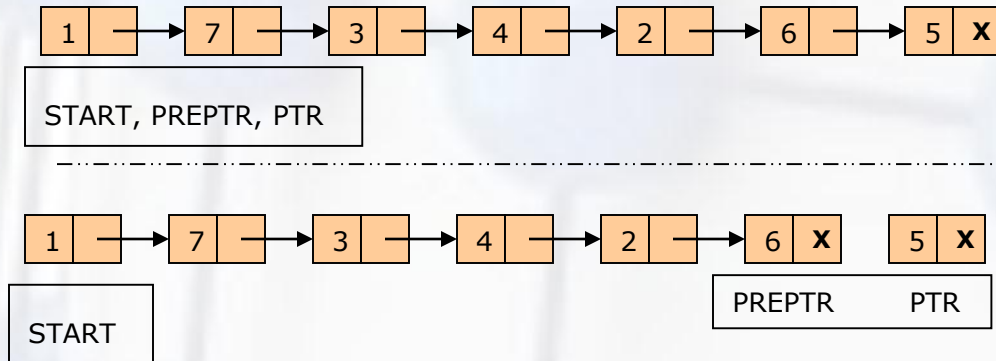
C++ Code

- `#include <iostream>`
- `using namespace std;`
-
- `struct Node {`
- `int data;`
- `Node* next;`
- `};`
-
- `// Return node back to AVAIL list`
- `void freeNode(Node*& AVAIL, Node* node) {`
- `node->next = AVAIL;`
- `AVAIL = node;`
- `}`
-
- `// Delete first node and return it to AVAIL`
- `void deleteFirst(Node*& START, Node*& AVAIL) {`
- `if (START == NULL) {`
- `cout << "UNDERFLOW" << endl;`
- `return;`
- `}`
-
- `Node* PTR = START; // Step 2`
- `START = START->next; // Step 3`
- `freeNode(AVAIL, PTR); // Step 4 (instead of delete)`
- `}`

- void display(Node* start) {
- Node* temp = start;
- while (temp != NULL) {
- cout << temp->data << " -> ";
- temp = temp->next;
- }
- cout << "NULL" << endl;
- }
-
- Node* initializeAvail(int n) {
- Node* head = NULL;
- for (int i = 0; i < n; i++) {
- Node* temp = new Node;
- temp->next = head;
- head = temp;
- }
- return head;
- }
-
- int main() {
- Node* AVAIL = initializeAvail(10); // free pool
- Node* START = NULL;
-
- // Take 3 nodes from AVAIL to build: 10 -> 20 -> 30
- Node* n1 = AVAIL; AVAIL = AVAIL->next; n1->data = 10;
- Node* n2 = AVAIL; AVAIL = AVAIL->next; n2->data = 20;
- Node* n3 = AVAIL; AVAIL = AVAIL->next; n3->data = 30;
- n1->next = n2; n2->next = n3; n3->next = NULL;
- START = n1;
-

- `cout << "Original List: ";`
- `display(START);`
-
- `deleteFirst(START, AVAIL);`
- `cout << "After deleting first node: ";`
- `display(START);`
-
- `deleteFirst(START, AVAIL);`
- `cout << "After deleting first node again: ";`
- `display(START);`
-
- `deleteFirst(START, AVAIL);`
- `cout << "After deleting last node: ";`
- `display(START);`
-
- `deleteFirst(START, AVAIL); // empty case`
- `}`

Deleting the Last Node



ALGORITHM TO DELETE THE LAST NODE OF THE LINKED LIST

Step 1: IF START = NULL, then

Write UNDERFLOW

Go to Step 8

[END OF IF]

Step 2: SET PTR = START

Step 3: Repeat Steps 4 and 5 while PTR->NEXT != NULL

Step 4: SET PREPTR = PTR

Step 5: SET PTR = PTR->NEXT

[END OF LOOP]

Step 6: SET PREPTR->NEXT = NULL

Step 7: FREE PTR

Step 8: EXIT

C++ CODE

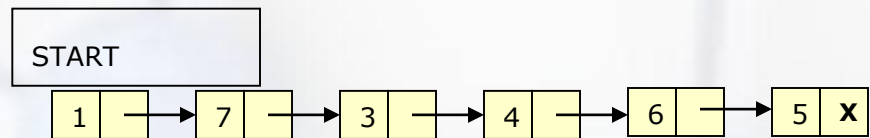
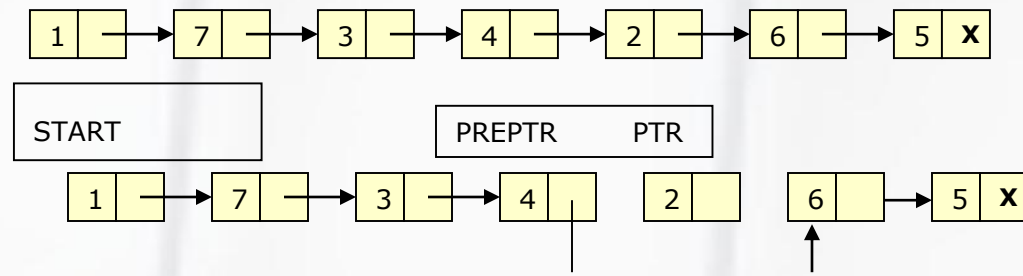
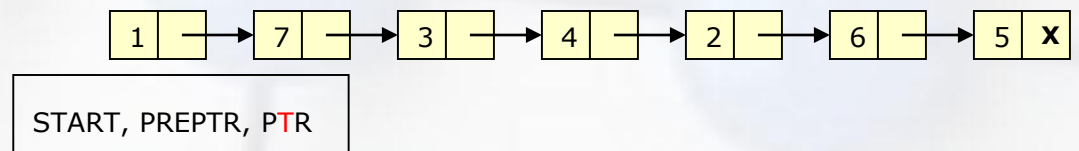
- `#include <iostream>`
- `using namespace std;`
-
- `struct Node {`
- `int data;`
- `Node* next;`
- `};`
-
- `// Return a deleted node back to AVAIL`
- `void freeNode(Node*& AVAIL, Node* node) {`
- `node->next = AVAIL;`
- `AVAIL = node;`
- `}`
-
- `// Delete the last node and return it to AVAIL`
- `void deleteLast(Node*& START, Node*& AVAIL) {`
- `// Step 1: Check if list is empty`
- `if (START == NULL) {`
- `cout << "UNDERFLOW" << endl;`
- `return;`
- `}`
-
- `// Case 1: Only one node in the list`
- `if (START->next == NULL) {`
- `Node* PTR = START;`
- `START = NULL;`
- `freeNode(AVAIL, PTR); // return to AVAIL`
- `return;`
- `}`

- // Case 2: More than one node
- Node* PTR = START;
- Node* PREPTR = NULL;
-
- // Traverse until PTR points to the last node
- while (PTR->next != NULL) {
- PREPTR = PTR; // Step 4
- PTR = PTR->next; // Step 5
- }
-
- // Disconnect last node
- PREPTR->next = NULL; // Step 6
-
- // Return last node to AVAIL instead of deleting
- freeNode(AVAIL, PTR); // Step 7
- }

- // Function to display linked list
- void display(Node* start) {
- Node* temp = start;
- while (temp != NULL) {
- cout << temp->data << " -> ";
- temp = temp->next;
- }
- cout << "NULL" << endl;
- }
-
- // Initialize AVAIL with free nodes
- Node* initializeAvail(int n) {
- Node* head = NULL;
- for (int i = 0; i < n; i++) {
- Node* temp = new Node;
- temp->next = head;
- head = temp;
- }
- return head;
- }

```
• int main() {
•     Node* AVAIL = initializeAvail(10); // free pool
•     Node* START = NULL;
•
•     // Build a list manually using AVAIL: 10 -> 20 -> 30 -> 40
•     Node* n1 = AVAIL; AVAIL = AVAIL->next; n1->data = 10;
•     Node* n2 = AVAIL; AVAIL = AVAIL->next; n2->data = 20;
•     Node* n3 = AVAIL; AVAIL = AVAIL->next; n3->data = 30;
•     Node* n4 = AVAIL; AVAIL = AVAIL->next; n4->data = 40;
•     n1->next = n2; n2->next = n3; n3->next = n4; n4->next = NULL;
•     START = n1;
•
•     cout << "Original List: ";
•     display(START);
•
•     // Delete last node
•     deleteLast(START, AVAIL);
•     cout << "After deleting last node: ";
•     display(START);
•
•     // Delete again
•     deleteLast(START, AVAIL);
•     cout << "After deleting again: ";
•     display(START);
•
•     // Delete until empty
•     deleteLast(START, AVAIL);
•     deleteLast(START, AVAIL);
•     cout << "After deleting all nodes: ";
•     display(START);
•
•     // Try deleting from empty list
•     deleteLast(START, AVAIL);
•
•     return 0;
• }
```


Deleting the Node After a Given Node



Deleting the Node After a Given Node

ALGORITHM TO DELETE THE NODE AFTER A GIVEN NODE FROM THE LINKED LIST

Step 1: IF START = NULL, then
 Write UNDERFLOW
 Go to Step 10

[END OF IF]

Step 2: SET PTR = START

Step 3: SET PREPTR = PTR

Step 4: Repeat Step 5 and 6 while PREPTR->DATA != NUM

Step 5: SET PREPTR = PTR

Step 6: SET PTR = PTR->NEXT

[END OF LOOP]

Step 7: SET TEMP = PTR->NEXT

Step 8: SET PREPTR->NEXT = TEMP->NEXT

Step 9: FREE TEMP

Step 10: EXIT

C++ CODE

- `#include <iostream>`
- `using namespace std;`
-
- `struct Node {`
- `int data;`
- `Node* next;`
- `};`
-
- `// Return node to AVAIL`
- `void freeNode(Node*& AVAIL, Node* node) {`
- `node->next = AVAIL;`
- `AVAIL = node;`
- `}`
-
- `// Delete a node with value NUM and return to AVAIL`
- `void deleteNode(Node*& START, Node*& AVAIL, int NUM) {`
- `if (START == NULL) {`
- `cout << "UNDERFLOW" << endl;`
- `return;`
- `}`

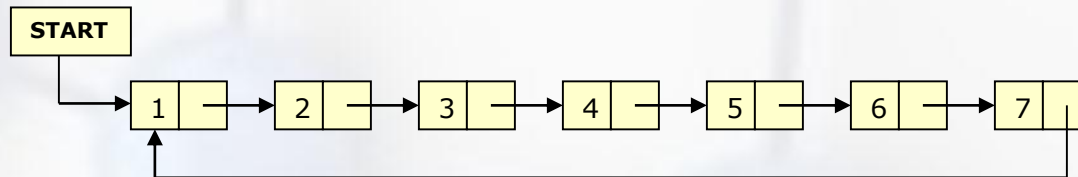
- // Case 1: First node contains NUM
- if (START->data == NUM) {
- Node* PTR = START;
- START = START->next;
- freeNode(AVAIL, PTR);
- return;
- }
-
- // Case 2: Search for NUM
- Node* PTR = START;
- Node* PREPTR = NULL;
-
- while (PTR != NULL && PTR->data != NUM) {
- PREPTR = PTR;
- PTR = PTR->next;
- }
-
- if (PTR == NULL) {
- cout << "VALUE NOT FOUND" << endl;
- return;
- }
-
- PREPTR->next = PTR->next;
- freeNode(AVAIL, PTR);
- }

- // Display
- void display(Node* start) {
- Node* temp = start;
- while (temp != NULL) {
- cout << temp->data << " -> ";
- temp = temp->next;
- }
- cout << "NULL" << endl;
- }
-
- // Initialize AVAIL with free nodes
- Node* initializeAvail(int n) {
- Node* head = NULL;
- for (int i = 0; i < n; i++) {
- Node* temp = new Node;
- temp->next = head;
- head = temp;
- }
- return head;
- }
-
- int main() {
- Node* AVAIL = initializeAvail(10);
- Node* START = NULL;
-
- // Build list using AVAIL: 10 -> 20 -> 30 -> 40
- Node* n1 = AVAIL; AVAIL = AVAIL->next; n1->data = 10;
- Node* n2 = AVAIL; AVAIL = AVAIL->next; n2->data = 20;
- Node* n3 = AVAIL; AVAIL = AVAIL->next; n3->data = 30;
- Node* n4 = AVAIL; AVAIL = AVAIL->next; n4->data = 40;
- n1->next = n2; n2->next = n3; n3->next = n4; n4->next = NULL;
- START = n1;

- `cout << "Original List: ";`
- `display(START);`
-
- `deleteNode(START, AVAIL, 30);`
- `cout << "After deleting 30: ";`
- `display(START);`
-
- `deleteNode(START, AVAIL, 10);`
- `cout << "After deleting 10: ";`
- `display(START);`
-
- `deleteNode(START, AVAIL, 99);`
- `deleteNode(START, AVAIL, 40);`
- `cout << "After deleting 40: ";`
- `display(START);`
-
- `deleteNode(START, AVAIL, 20);`
- `cout << "Final List: ";`
- `display(START);`
-
- `deleteNode(START, AVAIL, 5); // UNDERFLOW`
- `}`

Circular Linked List

- In a circular linked list, the last node contains a pointer to the first node of the list. We can have a circular singly linked list as well as circular doubly linked list. While traversing a circular linked list, we can begin at any node and traverse the list in any direction forward or backward until we reach the same node where we had started. Thus, a circular linked list has no beginning and no ending.



Circular Linked List

Algorithm to insert a new node in the beginning of **circular** the linked list

Step 1: IF AVAIL = NULL, then

 Write OVERFLOW

 Go to Step 7

[END OF IF]

Step 2: SET New_Node = AVAIL

Step 3: SET AVAIL = AVAIL->NEXT

Step 4: SET New_Node->DATA = VAL

Step 5: SET PTR = START

Step 6: Repeat Step 7 while PTR->NEXT != START

Step 7: PTR = PTR->NEXT

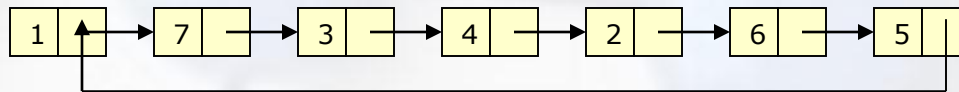
Step 8: SET New_Node->Next = START

Step 8: SET PTR->NEXT = New_Node

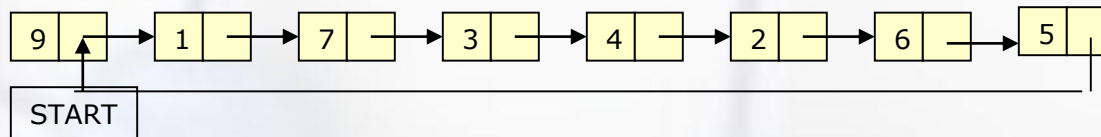
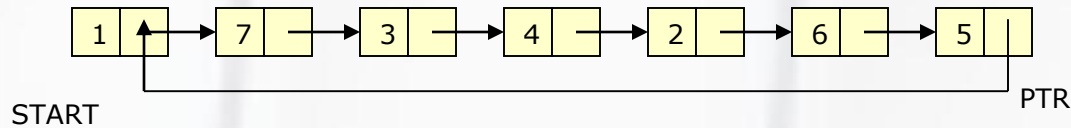
Step 6: SET START = New_Node

Step 7: EXIT

Circular Linked List



START, PTR



Circular Linked List

Algorithm to insert a new node at the end of the **circular** linked list

Step 1: IF AVAIL = NULL, then
 Write OVERFLOW
 Go to Step 7

[END OF IF]

Step 2: SET New_Node = AVAIL

Step 3: SET AVAIL = AVAIL->NEXT

Step 4: SET New_Node->DATA = VAL

Step 5: SET New_Node->Next = START

Step 6: SET PTR = START

Step 7: Repeat Step 8 while PTR->NEXT != START

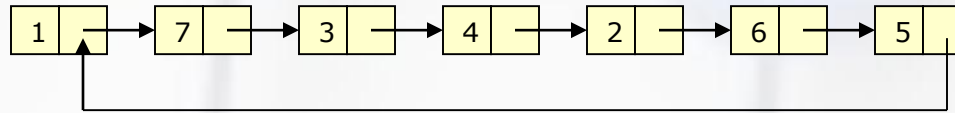
Step 8: SET PTR = PTR ->NEXT

[END OF LOOP]

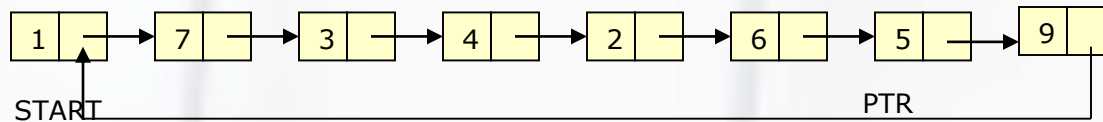
Step 9: SET PTR ->NEXT = New_Node

Step 10: EXIT

Circular Linked List



START, PTR



Circular Linked List

Algorithm to insert a new node after a node that has value NUM

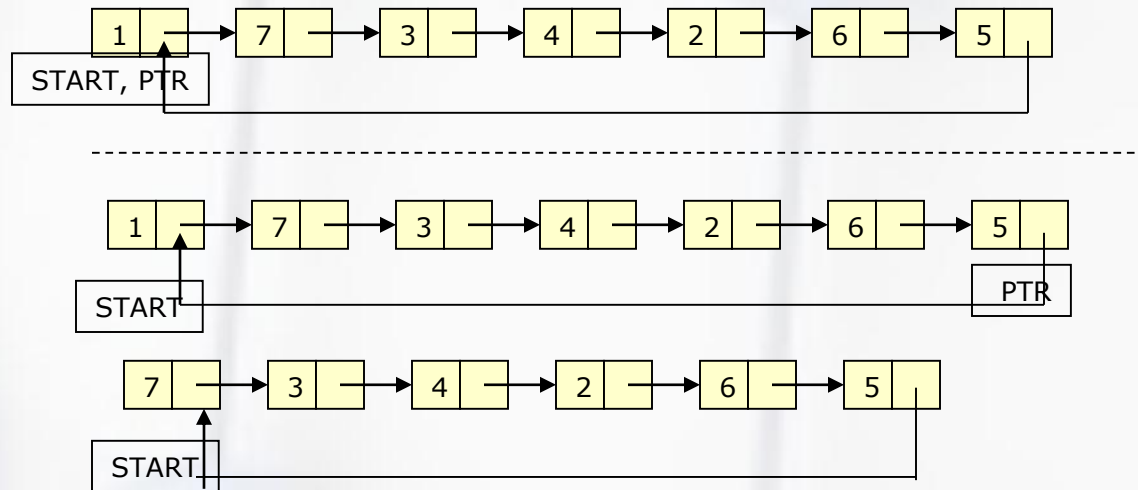
```
Step 1: IF AVAIL = NULL, then
        Write OVERFLOW
        Go to Step 12
    [END OF IF]
Step 2: SET New_Node = AVAIL
Step 3: SET AVAIL = AVAIL->NEXT
Step 4: SET New_Node->DATA = VAL
Step 5: SET PTR = START
Step 6: SET PREPTR = PTR
Step 7: Repeat Step 8 and 9 while PTR->DATA != NUM
Step 8:         SET PREPTR = PTR
Step 9:         SET PTR = PTR->NEXT
    [END OF LOOP]
Step 10: PREPTR->NEXT = New_Node
Step 11: SET New_Node->NEXT = PTR
Step 12: EXIT
```

Circular Linked List

Algorithm to delete the first node from the circular linked list

Step 1: IF START = NULL, then
 Write UNDERFLOW
 Go to Step 8
 [END OF IF]
Step 2: SET PTR = START
Step 3: Repeat Step 4 while PTR->NEXT != START
Step 4: SET PTR = PTR->NEXT
 [END OF IF]
Step 5: SET PTR->NEXT = START->NEXT
Step 6: FREE START
Step 7: SET START = PTR->NEXT
Step 8: EXIT

Circular Linked List



Circular Linked List

Algorithm to delete the last node of the **circular** linked list

Step 1: IF START = NULL, then
 Write UNDERFLOW
 Go to Step 8

 [END OF IF]

Step 2: SET PTR = START

Step 3: Repeat Step 4 while PTR->NEXT != START

Step 4: SET PREPTR = PTR

Step 5: SET PTR = PTR->NEXT

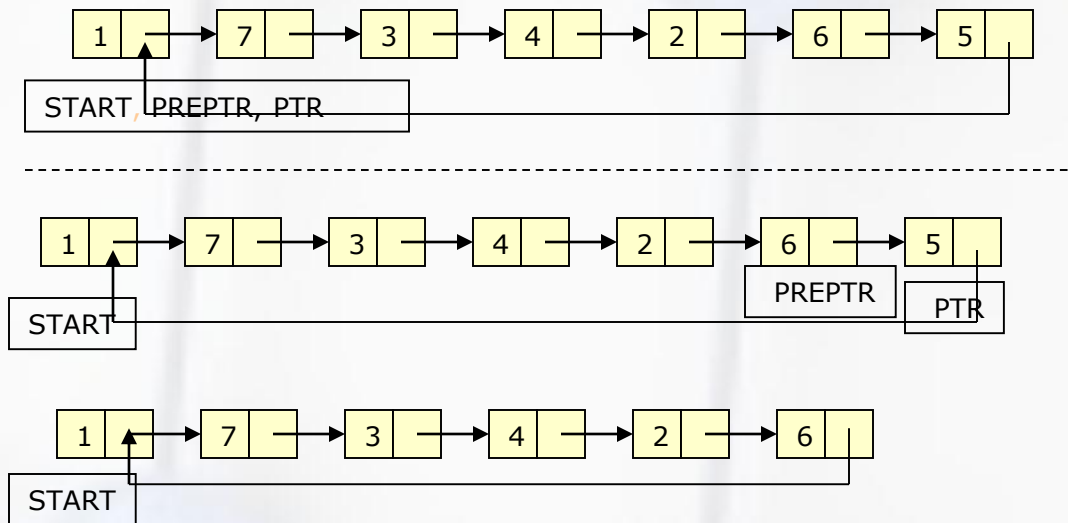
 [END OF LOOP]

Step 6: SET PREPTR->NEXT = START

Step 7: FREE PTR

Step 8: EXIT

Circular Linked List



Circular Linked List

Algorithm to delete the node after a given node from the circular linked list

Step 1: IF START = NULL, then

Write UNDERFLOW

Go to Step 9

[END OF IF]

Step 2: SET PTR = START

Step 3: SET PREPTR = PTR

Step 4: Repeat Step 5 and 6 while PREPTR->DATA != NUM

Step 5: SET PREPTR = PTR

Step 6: SET PTR = PTR->NEXT

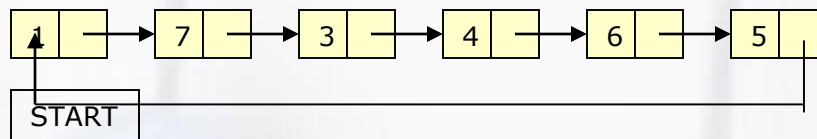
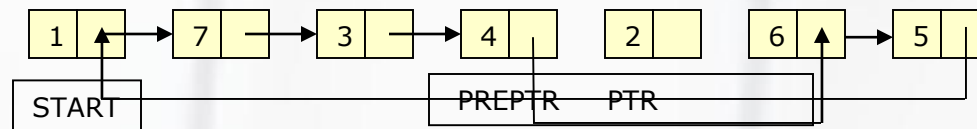
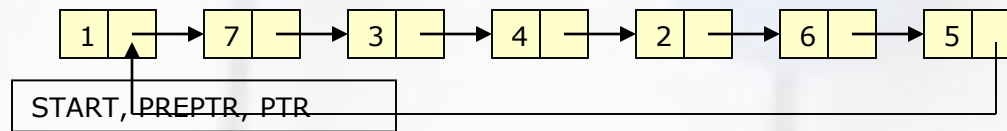
[END OF LOOP]

Step 7: SET PREPTR->NEXT = PTR->NEXT

Step 8: FREE PTR

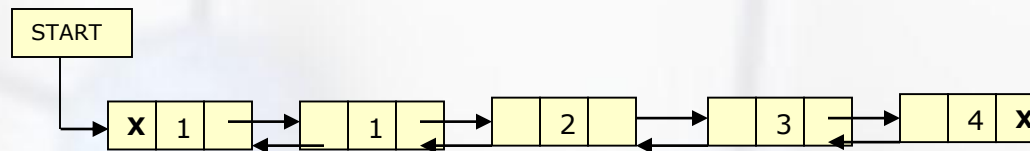
Step 9: EXIT

Circular Linked List



Doubly Linked List

- A doubly linked list or a two way linked list is a more complex type of linked list which contains a pointer to the next as well as previous node in the sequence. Therefore, it consists of three parts and not just two. The three parts are data, a pointer to the next node and a pointer to the previous node



Doubly Linked List

- In C language, the structure of a doubly linked list is given as,
struct node
{ struct node *prev;
 int data;
 struct node *next;
};
- The prev field of the first node and the next field of the last node will contain NULL. The prev field is used to store the address of the preceding node. This would enable to traverse the list in the backward direction as well.

Doubly Linked List

Algorithm to insert a new node in the beginning of the doubly linked list

Step 1: IF AVAIL = NULL, then
Write OVERFLOW
Go to Step 8

[END OF IF]

Step 2: SET New_Node = AVAIL

Step 3: SET AVAIL = AVAIL->NEXT

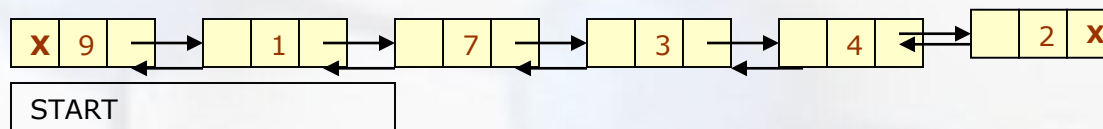
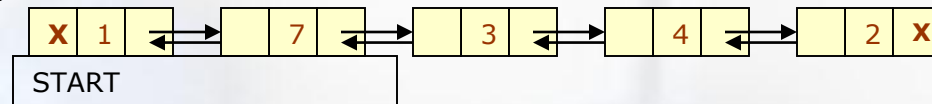
Step 4: SET New_Node->DATA = VAL

Step 5: SET New_Node->PREV = NULL

Step 6: SET New_Node->Next = START

Step 7: SET START = New_Node

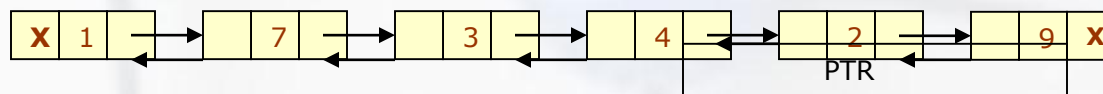
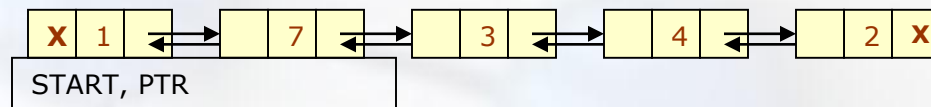
Step 8: EXIT



Doubly Linked List

Algorithm to insert a new node at the end of the doubly linked list

```
Step 1: IF AVAIL = NULL, then
        Write OVERFLOW
        Go to Step 11
    [END OF IF]
Step 2: SET New_Node = AVAIL
Step 3: SET AVAIL = AVAIL->NEXT
Step 4: SET New_Node->DATA = VAL
Step 5: SET New_Node->Next = NULL
Step 6: SET PTR = START
Step 7: Repeat Step 8 while PTR->NEXT != NULL
Step 8:     SET PTR = PTR->NEXT
    [END OF LOOP]
Step 9: SET PTR->NEXT = New_Node
Step 10: New_Node->PREV = PTR
Step 11: EXIT
```

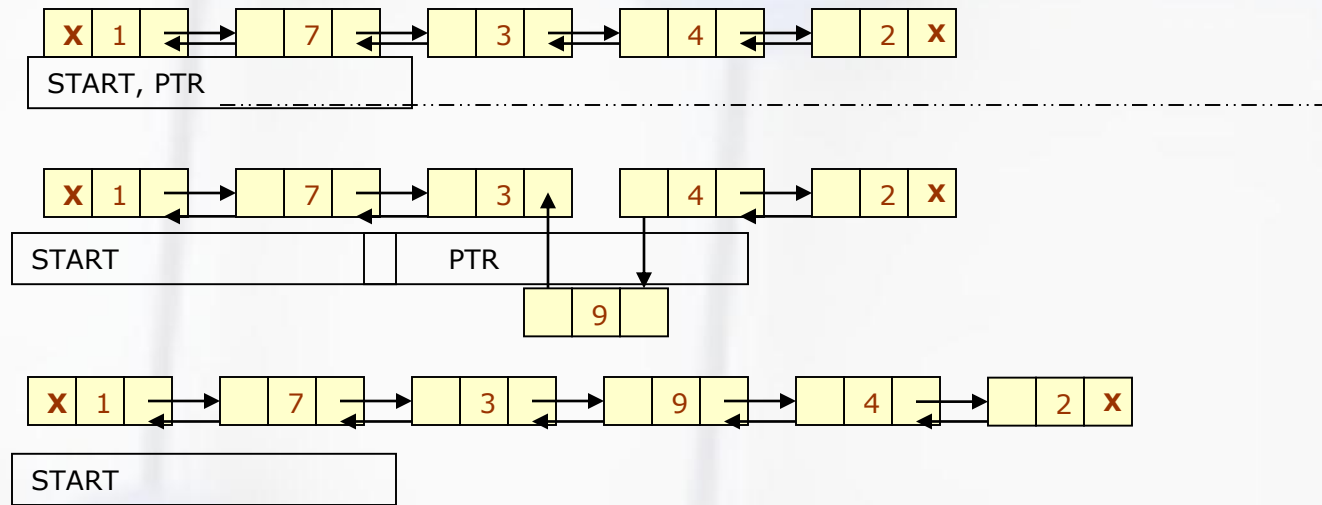


Doubly Linked List

Algorithm to insert a new node after a node that has value NUM

```
Step 1: IF AVAIL = NULL, then
        Write OVERFLOW
        Go to Step 11
    [END OF IF]
Step 2: SET New_Node = AVAIL
Step 3: SET AVAIL = AVAIL->NEXT
Step 4: SET New_Node->DATA = VAL
Step 5: SET PTR = START
Step 6: Repeat Step 8 while PTR->DATA != NUM
Step 7:         SET PTR = PTR->NEXT
    [END OF LOOP]
Step 8: New_Node->NEXT = PTR->NEXT
Step 9: SET New_Node->PREV = PTR
Step 10: SET PTR->NEXT = New_Node
Step 11: EXIT
```

Doubly Linked List



Doubly Linked List

Algorithm to delete the first node from the doubly linked list

Step 1: IF $START = NULL$, then
 Write UNDERFLOW
 Go to Step 6
[END OF IF]

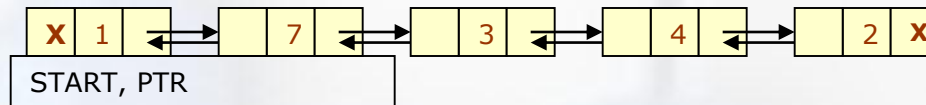
Step 2: SET $PTR = START$

Step 3: SET $START = START \rightarrow NEXT$

Step 4: SET $START \rightarrow PREV = NULL$

Step 5: FREE PTR

Step 6: EXIT



Doubly Linked List

Algorithm to delete the last node of the doubly linked list

Step 1: IF START = NULL, then

Write UNDERFLOW

Go to Step 7

[END OF IF]

Step 2: SET PTR = START

Step 3: Repeat Step 4 and 5 while PTR->NEXT != NULL

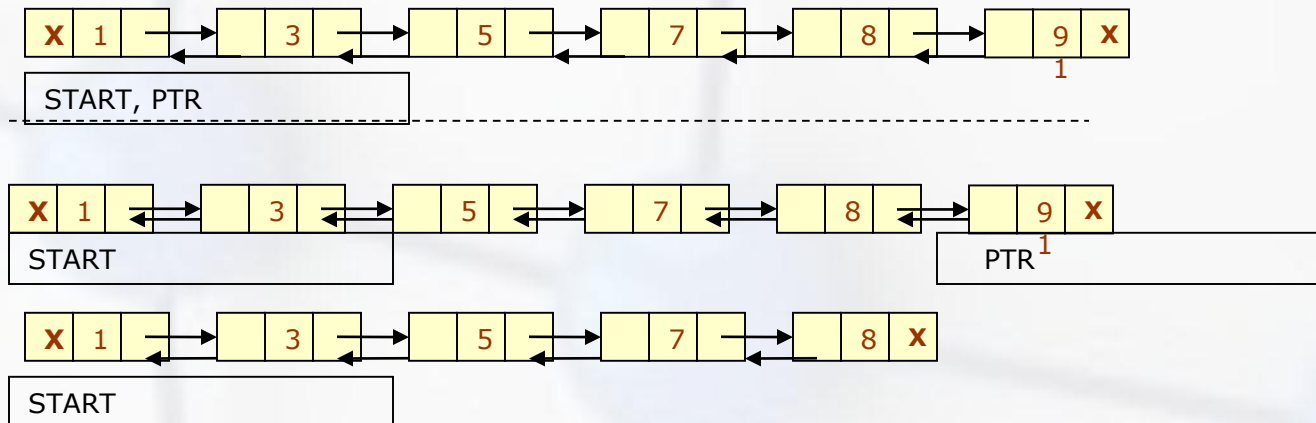
Step 4: SET PTR = PTR->NEXT

[END OF LOOP]

Step 5: SET PTR->PREV->NEXT = NULL

Step 6: FREE PTR

Step 7: EXIT

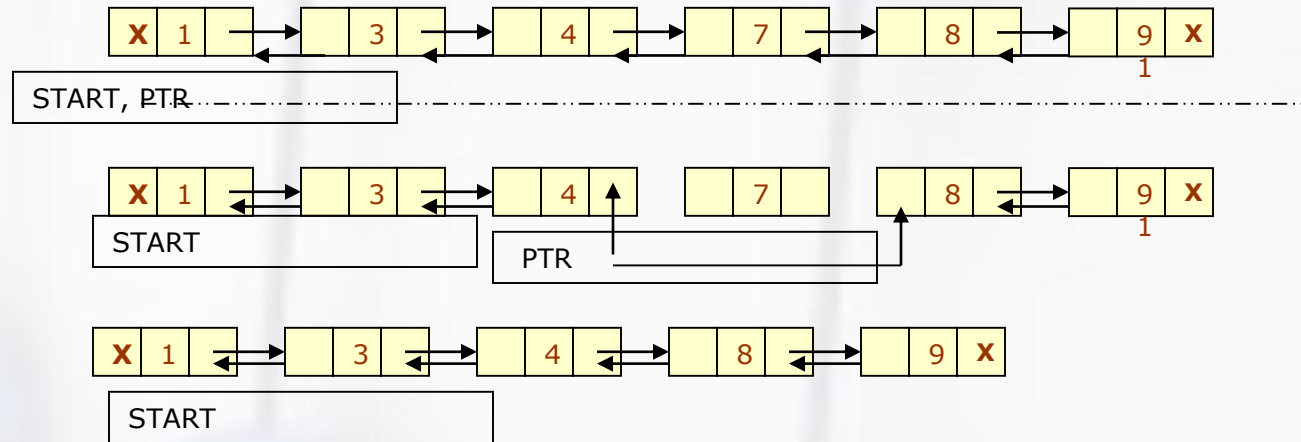


Doubly Linked List

Algorithm to delete the node after a given node from the doubly linked list

```
Step 1: IF START = NULL, then
        Write UNDERFLOW
        Go to Step 9
    [END OF IF]
Step 2: SET PTR = START
Step 3: Repeat Step 4 while PTR->DATA != NUM
Step 4:         SET PTR = PTR->NEXT
    [END OF LOOP]
Step 5: SET TEMP = PTR->NEXT
Step 6: SET PTR->NEXT = TEMP->NEXT
Step 7: SET TEMP->NEXT->PREV = PTR
Step 8: FREE TEMP
Step 9: EXIT
```

Doubly Linked List

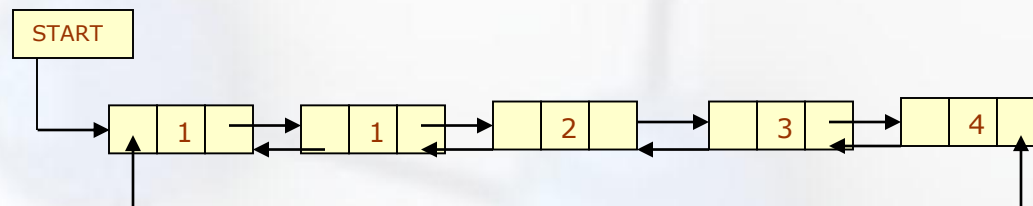


5. Circular Doubly Linked List

- A circular doubly linked list or a circular two way linked list is a more complex type of linked list which contains a pointer to the next as well as previous node in the sequence.
- The difference between a doubly linked and a circular doubly linked list is same as that exists between a singly linked list and a circular linked list. The circular doubly linked list does not contain NULL in the previous field of the first node and the next field of the last node. Rather, the next field of the last node stores the address of the first node of the list, i.e; START. Similarly, the previous field of the first field stores the address of the last node.

Circular Doubly Linked List

- Since a circular doubly linked list contains three parts in its structure, it calls for more space per node and for more expensive basic operations. However, it provides the ease to manipulate the elements of the list as it maintains pointers to nodes in both the directions. The main advantage of using a circular doubly linked list is that it makes searches twice as efficient.



Circular Doubly Linked List

Algorithm to insert a new node in the beginning of the circular doubly linked list

Step 1: IF AVAIL = NULL, then
 Write OVERFLOW
 Go to Step 12
[END OF IF]

Step 2: SET New_Node = AVAIL

Step 3: SET AVAIL = AVAIL->NEXT

Step 4: SET New_Node->DATA = VAL

Step 6: SET START->PREV->NEXT = new_node;

Step 7: SET New_Node->PREV = START->PREV;

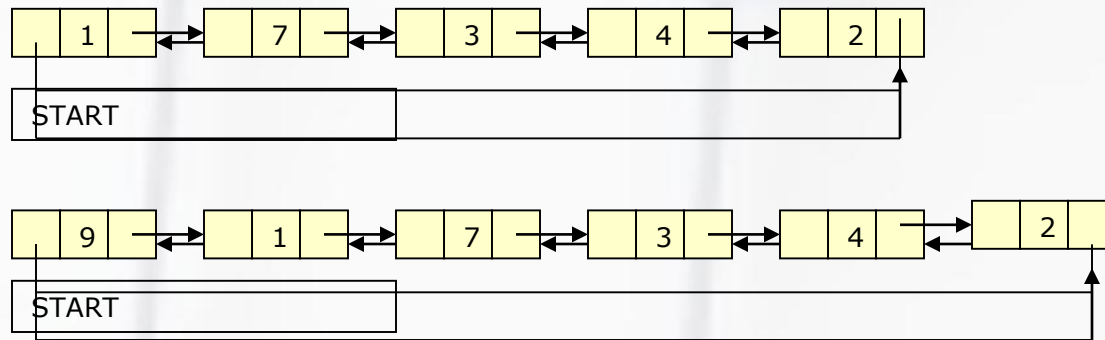
Step 8: SET START->PREV = new_Node;

Step 9: SET new_node->next = START;

Step 10: SET START = New_Node

Step 11: EXIT

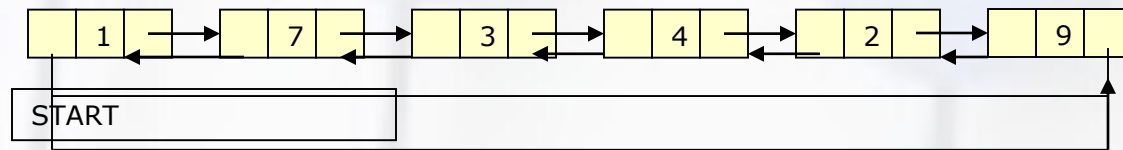
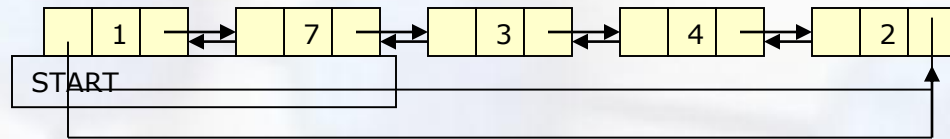
Circular Doubly Linked List



Circular Doubly Linked List

Algorithm to insert a new node at the end of the **circular doubly** linked list

- Step 1: IF AVAIL = NULL, then
 - Write OVERFLOW
 - Go to Step 13
 - [END OF IF]
- Step 2: SET New_Node = AVAIL
- Step 3: SET AVAIL = AVAIL->NEXT
- Step 4: SET New_Node->DATA = VAL
- Step 5: SET New_Node->Next = START
- Step 6: SET PTR = START
- Step 7: Repeat Step 8 while PTR->NEXT != START
- Step 8: SET PTR = PTR ->NEXT
 - [END OF LOOP]
- Step 9: SET PTR ->NEXT = New_Node
- Step 10: SET New_Node → PREV = PTR
- Step 11: SET New_Node → NEXT = START
- Step 12: SET START → PREV = New_Node
- Step 13: EXIT

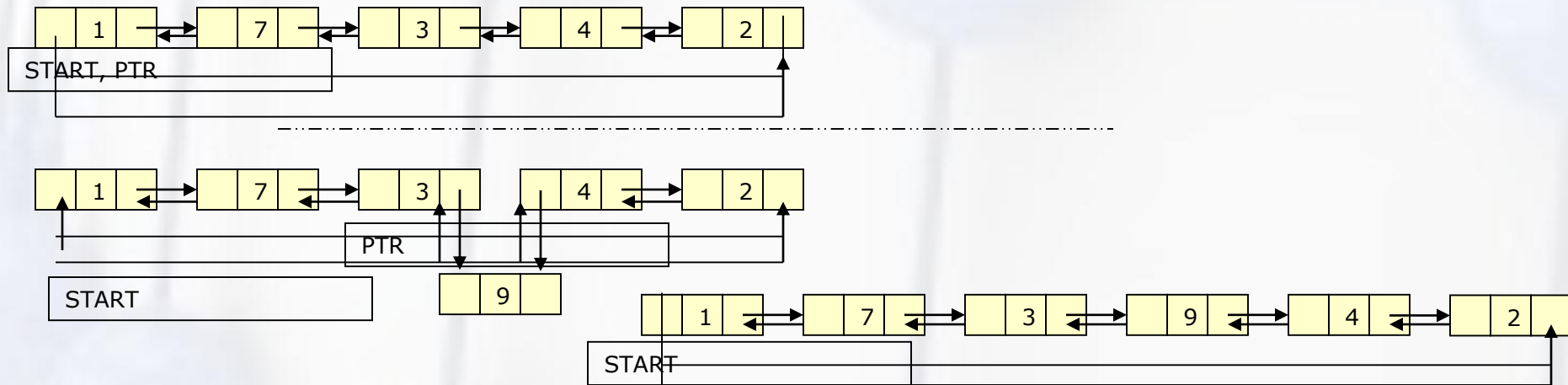


Circular Doubly Linked List

Algorithm to insert a new node after a node that has value NUM

```
Step 1: IF AVAIL = NULL, then
        Write OVERFLOW
        Go to Step 11
    [END OF IF]
Step 2: SET New_Node = AVAIL
Step 3: SET AVAIL = AVAIL->NEXT
Step 4: SET New_Node->DATA = VAL
Step 5: SET PTR = START
Step 6: Repeat Step 8 while PTR->DATA != NUM
Step 7:         SET PTR = PTR->NEXT
    [END OF LOOP]
Step 8: New_Node->NEXT = PTR->NEXT
Step 9: SET PTR->NEXT->PREV = New_Node
Step 9: SET New_Node->PREV = PTR
Step 10: SET PTR->NEXT = New_Node
Step 11: EXIT
```

Circular Doubly Linked List



Circular Doubly Linked List

Algorithm to delete the first node from the **circular doubly linked list**

Step 1: IF $START = NULL$, then
 Write UNDERFLOW
 Go to Step 8

[END OF IF]

Step 2: SET $PTR = START$

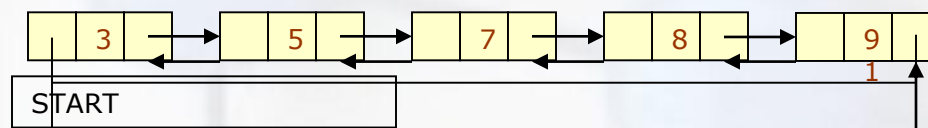
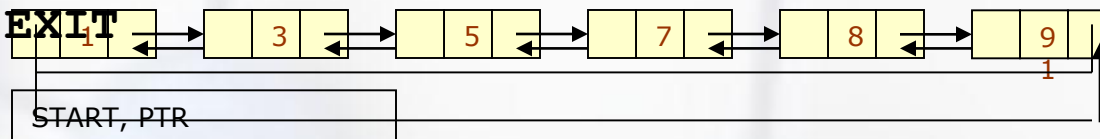
Step 3: SET $PTR \rightarrow PREV \rightarrow NEXT = PTR \rightarrow NEXT$

Step 4: SET $PTR \rightarrow NEXT \rightarrow PREV = PTR \rightarrow PREV$

Step 5: SET $START = START \rightarrow NEXT$

Step 6: FREE PTR

Step 7: **EXIT**



Circular Doubly Linked List

Algorithm to delete the last node of the **circular doubly linked list**

Step 1: IF $START = NULL$, then
Write UNDERFLOW
Go to Step 8

[END OF IF]

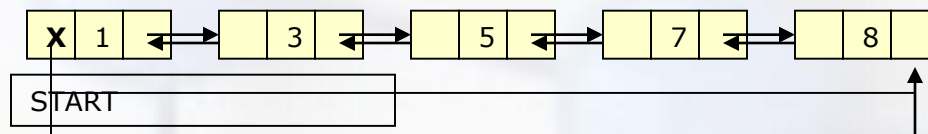
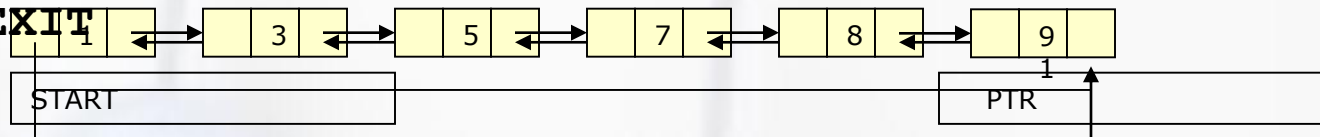
Step 2: SET $PTR = START \rightarrow PREV$

Step 5: SET $PTR \rightarrow PREV \rightarrow NEXT = START$

Step 6: SET $START \rightarrow PREV = PTR \rightarrow PREV$

Step 7: FREE PTR

Step 8: EXIT

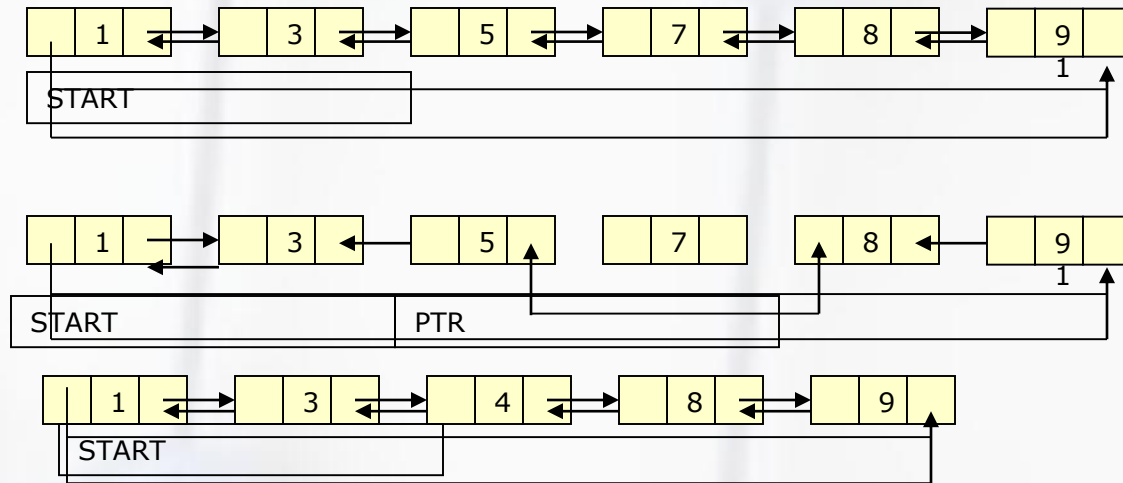


Circular Doubly Linked List

Algorithm to delete the node after a given node from the **circular doubly** linked list

```
Step 1: IF START = NULL, then
        Write UNDERFLOW
        Go to Step 9
    [END OF IF]
Step 2: SET PTR = START
Step 3: Repeat Step 4 while PTR->DATA != NUM
Step 4:         SET PTR = PTR->NEXT
    [END OF LOOP]
Step 5: SET TEMP = PTR->NEXT
Step 6: SET PTR->NEXT = TEMP->NEXT
Step 7: SET TEMP->NEXT->PREV = PTR
Step 8: FREE TEMP
Step 9: EXIT
```

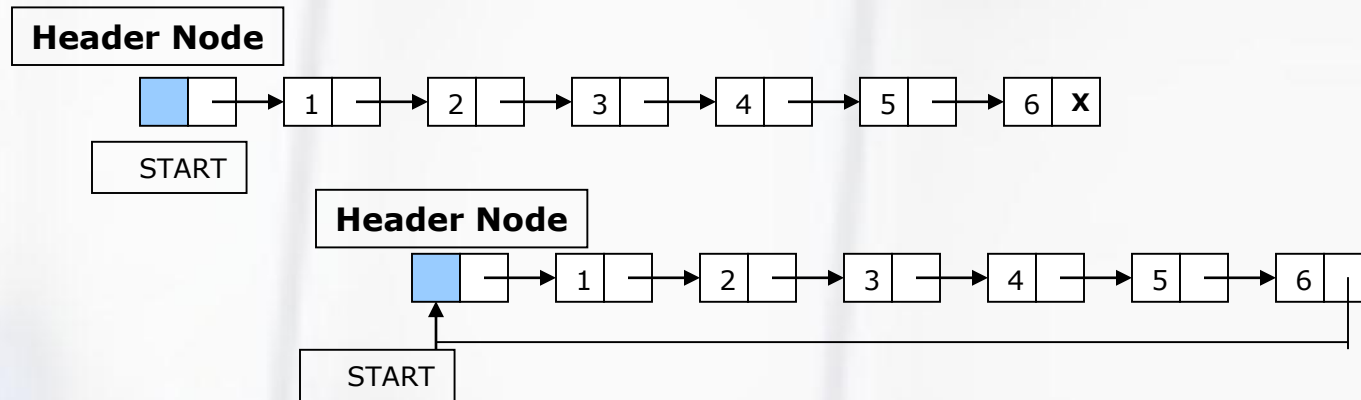
Circular Doubly Linked List



Circular Doubly Linked List

- A header linked list is a special type of linked list which contains a header node at the beginning of the list. So, in a header linked list START will not point to the first node of the list but START will contain the address of the header node. There are basically two variants of a header linked list-
- Grounded header linked list which stores NULL in the next field of the last node
- Circular header linked list which stores the address of the header node in the next field of the last node. Here, the header node will denote the end of the list.

Circular Doubly Linked List



Circular Doubly Linked List

Algorithm to traverse a Circular Header Linked List

Step 1: SET PTR = START->NEXT

Step 2: Repeat Steps 3 and 4 while PTR != START

Step 3: Apply PROCESS to PTR->DATA

Step 4: SET PTR = PTR->NEXT

 [END OF LOOP]

Step 5: EXIT

Circular Doubly Linked List

Algorithm to insert a new node after a given node

```
Step 1: IF AVAIL = NULL, then
        Write OVERFLOW
        Go to Step 10
    [END OF IF]
Step 2: SET New_Node = AVAIL
Step 3: SET AVAIL = AVAIL->NEXT
Step 4: SET PTR = START->NEXT
Step 5: SET New_Node->DATA = VAL
Step 6: Repeat step 4 while PTR->DATA != NUM
Step 7:     SET PTR = PTR->NEXT
    [END OF LOOP]
Step 8: New_Node->NEXT = PTR->NEXT
Step 9: SET PTR->NEXT = New_Node
Step 10: EXIT
```